

Project Documentation: Architecture, Distillation & Optimization

Technical Documentation

November 27, 2025

Contents

1	The Student Architecture (Model Backbone)	2
1.1	Root Mean Square Normalization (RMSNorm)	2
1.1.1	Definition	2
1.1.2	Mathematical Derivation	2
1.1.3	Code Implementation	2
1.2	Rotary Positional Embeddings (RoPE)	2
1.2.1	Definition	2
1.2.2	Mathematical Derivation	2
1.2.3	Code Implementation	3
1.3	SwiGLU Activation Mechanism	3
1.3.1	Definition	3
1.3.2	Mathematical Derivation	3
1.3.3	Code Implementation	4
1.4	Grouped Query Attention (GQA)	4
1.4.1	Definition	4
1.4.2	Mathematical Derivation	4
1.4.3	Code Implementation	4
2	The Distillation Methodology (The Brain)	5
2.1	Kullback-Leibler (KL) Divergence	5
2.1.1	Definition	5
2.1.2	Mathematical Derivation	5
2.1.3	Code Implementation	5
2.2	Multi-Teacher Logit Aggregation	5
2.2.1	Definition	5
2.2.2	Mathematical Derivation	6
2.2.3	Code Implementation	6
3	Optimization & Training Dynamics	6
3.1	Adaptive Optimization (AdamW)	6
3.1.1	Definition	6
3.1.2	Mathematical Derivation	6
3.1.3	Code Implementation	7
3.2	Cosine Annealing Schedule	7
3.2.1	Definition	7
3.2.2	Mathematical Derivation	7
3.2.3	Code Implementation	7

1 The Student Architecture (Model Backbone)

File Focus: `student_architecture.py`

1.1 Root Mean Square Normalization (RMSNorm)

1.1.1 Definition

RMSNorm is a normalization technique used in modern Transformer architectures (like Llama and Gemma) to stabilize training. Unlike Layer Normalization, which re-centers and re-scales the input (subtracting the mean and dividing by standard deviation), RMSNorm assumes the mean is negligible and only re-scales the input based on its Root Mean Square (RMS) value. This reduces computational overhead while preserving model convergence properties.

1.1.2 Mathematical Derivation

The Objective: Given an input vector $x \in \mathbb{R}^d$, we want to normalize it to prevent activation explosion in deep networks.

The Metric (RMS): Instead of calculating variance (which requires the mean μ), we calculate the Root Mean Square magnitude:

$$\text{RMS}(x) = \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \epsilon} \quad (1)$$

(where ϵ is a small constant for numerical stability).

The Transformation: The vector is normalized by dividing by its RMS value and then scaled by a learnable parameter γ (gain):

$$\bar{x}_i = \frac{x_i}{\text{RMS}(x)} \cdot \gamma_i \quad (2)$$

Gradient Advantage: The derivative of RMSNorm is simpler than LayerNorm because it eliminates the dependency on the mean μ , forcing the gradient to scale linearly with the input.

1.1.3 Code Implementation

```
1 class RMSNorm(nn.Module):
2     def __init__(self, dim: int, eps: float = 1e-5):
3         super().__init__()
4         self.eps = eps
5         self.weight = nn.Parameter(torch.ones(dim))
6
7     def forward(self, x: torch.Tensor) -> torch.Tensor:
8         variance = x.to(torch.float32).pow(2).mean(-1, keepdim=True)
9         x_normed = x * torch.rsqrt(variance + self.eps)
10        return self.weight * x_normed.to(x.dtype)
```

1.2 Rotary Positional Embeddings (RoPE)

1.2.1 Definition

Rotary Positional Embeddings (RoPE) encode positional information by rotating the Query (q) and Key (k) vectors in a high-dimensional space. Unlike additive position embeddings (which simply add a vector p to x), RoPE applies a multiplicative rotation matrix. This mathematically guarantees that the attention score between two tokens depends only on their relative distance ($m - n$), rather than their absolute positions m and n .

1.2.2 Mathematical Derivation

Complex Representation: We treat pairs of adjacent features in the embedding vector as complex numbers. Let a 2D vector at position m be x_m .

$$x_m = x \cdot e^{im\theta} \quad (3)$$

The Attention Score (Dot Product): The relevance of a query at position m to a key at position n is the inner product (real part of the complex product):

$$\begin{aligned}\langle q_m, k_n \rangle &= \text{Re}(q_m \cdot k_n^*) \\ &= \text{Re}((q e^{im\theta}) \cdot (k e^{in\theta})^*) \\ &= \text{Re}(q \cdot k \cdot e^{i(m-n)\theta})\end{aligned}$$

This proves the score relies solely on the relative distance $(m - n)$.

Matrix Form: In $\mathbb{R}^d$, this operation is equivalent to multiplying by a block-diagonal rotation matrix:

$$\begin{pmatrix} x'_1 \\ x'_2 \end{pmatrix} = \begin{pmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \quad (4)$$

1.2.3 Code Implementation

```
1 class RotaryEmbedding(nn.Module):
2     def __init__(self, dim: int, max_position_embeddings: int = 2048, base:
3         float = 10000.0):
4         super().__init__()
5         inv_freq = 1.0 / (base ** (torch.arange(0, dim, 2).float() / dim))
6         self.register_buffer("inv_freq", inv_freq, persistent=False)
7
8     def rotate_half(x: torch.Tensor) -> torch.Tensor:
9         x1, x2 = x.chunk(2, dim=-1)
10        return torch.cat((-x2, x1), dim=-1)
11
12    def apply_rotary_pos_emb(q, k, cos, sin):
13        q_embed = (q * cos) + (rotate_half(q) * sin)
14        k_embed = (k * cos) + (rotate_half(k) * sin)
15        return q_embed, k_embed
```

1.3 SwiGLU Activation Mechanism

1.3.1 Definition

SwiGLU is a specialized activation function used in the Feed-Forward Network (FFN) layers of LLMs. It combines the Swish activation function with a Gated Linear Unit (GLU) structure. This mechanism allows the network to control the flow of information via a learned "gate" while preserving gradient magnitude better than ReLU.

1.3.2 Mathematical Derivation

Swish Activation: First, we define the Swish function, which is a smooth, non-monotonic approximation of ReLU:

$$\text{Swish}(x) = x \cdot \sigma(x) = \frac{x}{1 + e^{-x}} \quad (5)$$

Gated Linear Unit (GLU): A GLU splits the input into two paths: a "content" path and a "gate" path.

$$\text{GLU}(x, W, V) = (xW) \otimes \sigma(xV) \quad (6)$$

SwiGLU Formulation: We project the input x into three matrices: Gate (W_g), Up (W_u), and Down (W_d).

$$\begin{aligned}x_{gate} &= \text{Swish}(xW_g) \\ x_{up} &= xW_u \\ y &= (x_{gate} \otimes x_{up})W_d\end{aligned}$$

1.3.3 Code Implementation

```
1 class SwiGLU(nn.Module):
2     def __init__(self, dim: int, hidden_dim: int):
3         super().__init__()
4         self.gate_proj = nn.Linear(dim, hidden_dim, bias=False)
5         self.up_proj = nn.Linear(dim, hidden_dim, bias=False)
6         self.down_proj = nn.Linear(hidden_dim, dim, bias=False)
7
8     def forward(self, x: torch.Tensor) -> torch.Tensor:
9         gate = F.silu(self.gate_proj(x))
10        up = self.up_proj(x)
11        return self.down_proj(gate * up)
```

1.4 Grouped Query Attention (GQA)

1.4.1 Definition

Grouped Query Attention (GQA) is an optimization technique that interpolates between Multi-Head Attention (MHA) and Multi-Query Attention (MQA). By assigning a single KV head to a group of Query heads, it reduces memory usage by a factor of G while maintaining model performance.

1.4.2 Mathematical Derivation

Replication Factor: We define the replication factor R as the ratio of query heads to KV heads:

$$R = \frac{H_q}{H_{kv}} \quad (7)$$

Broadcasting (Tensor Expansion): To perform the dot product QK^T , the dimensions must align. We "repeat" the Key (K) and Value (V) tensors R times along the head dimension.

$$K_{expanded} = \text{Repeat}(K, \text{times} = R) \quad (8)$$

Attention Operation:

$$\text{Attention}(Q, K_{expanded}, V_{expanded}) = \text{softmax}\left(\frac{QK_{expanded}^T}{\sqrt{d_k}}\right)V_{expanded} \quad (9)$$

1.4.3 Code Implementation

```
1 class GroupedQueryAttention(nn.Module):
2     def _repeat_kv(self, hidden_states: torch.Tensor, n_rep: int) -> torch.
3     Tensor:
4         if n_rep == 1:
5             return hidden_states
6
7         batch, num_kv_heads, seq_len, head_dim = hidden_states.shape
8
9         hidden_states = hidden_states[:, :, None, :, :].expand(
10            batch, num_kv_heads, n_rep, seq_len, head_dim
11        )
12
13        return hidden_states.reshape(batch, num_kv_heads * n_rep, seq_len,
14            head_dim)
15
16    def forward(self, ...):
17        key_states = self._repeat_kv(key_states, self.num_key_value_groups)
18        value_states = self._repeat_kv(value_states, self.num_key_value_groups)
```

2 The Distillation Methodology (The Brain)

File Focus: `losses.py`, `direct_train.py`

2.1 Kullback-Leibler (KL) Divergence

2.1.1 Definition

Kullback-Leibler (KL) Divergence measures the information loss when a student model's probability distribution Q is used to approximate a teacher model's distribution P . It uses "soft" targets (probabilities), allowing the student to learn nuanced relationships between classes.

2.1.2 Mathematical Derivation

Softmax with Temperature: Raw logits z are converted into probabilities using a temperature parameter T .

$$P_i(z) = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)} \quad (10)$$

The Divergence Formula: KL Divergence is the difference between Cross-Entropy and Entropy:

$$D_{KL}(P||Q) = H(P, Q) - H(P) = \sum_x P(x) \log \left(\frac{P(x)}{Q(x)} \right) \quad (11)$$

Gradient Scaling: To keep gradients consistent when using temperature T , we multiply the final loss by T^2 .

$$\mathcal{L}_{distill} = T^2 \cdot D_{KL}(P_{soft}||Q_{soft}) \quad (12)$$

2.1.3 Code Implementation

```
1 class DirectMultiTeacherDistillationLoss(nn.Module):
2     def __init__(self, temperature: float = 2.0, ...):
3         super().__init__()
4         self.temperature = temperature
5
6     def forward(self, student_logits, teacher_logits_list, labels):
7         student_log_probs = F.log_softmax(shift_student_logits / self.
8         temperature, dim=-1)
9         teacher_probs = F.softmax(shift_teacher_logits / self.temperature, dim
10        =-1)
11
12        distill_loss = F.kl_div(
13            student_log_probs,
14            teacher_probs,
15            reduction='none'
16        ).sum(dim=-1)
17
18        distill_loss = distill_loss * (self.temperature ** 2)
19
20        return { 'distill_loss': distill_loss, ... }
```

2.2 Multi-Teacher Logit Aggregation

2.2.1 Definition

In Direct Multi-Teacher Distillation, the student learns from an ensemble of teachers. Their outputs (logits) are aggregated to form a superior target distribution, based on the "Wisdom of Crowds" principle.

2.2.2 Mathematical Derivation

Linear Aggregation: We aggregate the raw logits using a weighted average. Let α_k be the weight for Teacher k .

$$Z_{\text{aggregated}} = \sum_{k=1}^N \alpha_k \cdot Z_k(x) \quad (13)$$

Real-Time Forward Pass: $Z_k(x)$ is computed dynamically at step t :

$$Z_k^{(t)} = f_k(x^{(t)}; \theta_k) \quad (14)$$

2.2.3 Code Implementation

```

1 @torch.no_grad()
2 def get_teacher_logits(self, input_ids, attention_mask):
3     teacher_logits_list = []
4     for teacher in self.teachers:
5         outputs = teacher(input_ids=input_ids, attention_mask=attention_mask)
6         teacher_logits_list.append(outputs.logits)
7     return teacher_logits_list
8
9 def forward(self, student_logits, teacher_logits_list, labels):
10    if self.teacher_weights is not None:
11        teacher_logits = sum(
12            w * logits for w, logits in zip(self.teacher_weights,
13            teacher_logits_list)
14        )
15    else:
16        teacher_logits = sum(teacher_logits_list) / len(teacher_logits_list)

```

3 Optimization & Training Dynamics

File Focus: direct_train.py

3.1 Adaptive Optimization (AdamW)

3.1.1 Definition

AdamW decouples weight decay from the gradient update. Unlike standard Adam, which adds an L_2 term to the loss (coupling decay with adaptive steps), AdamW applies weight decay directly to the weights.

3.1.2 Mathematical Derivation

Moment Estimation:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (15)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (16)$$

Bias Correction:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (17)$$

The Decoupled Update Rule:

$$\theta_t = \theta_{t-1} - \eta \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda \theta_{t-1} \right) \quad (18)$$

Here, $\lambda \theta_{t-1}$ is the decoupled weight decay.

3.1.3 Code Implementation

```
1 def _create_optimizer(self) -> AdamW:
2     no_decay = ['bias', 'LayerNorm.weight', 'layer_norm.weight', 'norm.weight']
3
4     optimizer_grouped_parameters = [
5         {
6             'params': [
7                 p for n, p in self.student.named_parameters()
8                 if not any(nd in n for nd in no_decay) and p.requires_grad
9             ],
10            'weight_decay': self.config.weight_decay,
11        },
12        {
13            'params': [
14                p for n, p in self.student.named_parameters()
15                if any(nd in n for nd in no_decay) and p.requires_grad
16            ],
17            'weight_decay': 0.0,
18        },
19    ]
20
21    return AdamW(
22        optimizer_grouped_parameters,
23        lr=self.config.learning_rate,
24        betas=(self.config.adam_beta1, self.config.adam_beta2),
25        eps=self.config.adam_epsilon,
26    )
```

3.2 Cosine Annealing Schedule

3.2.1 Definition

The learning rate schedule controls the step size η during training. We use Cosine Annealing with Warmup to ensure smooth convergence.

3.2.2 Mathematical Derivation

Warmup Phase ($t < T_{warmup}$):

$$\eta_t = \eta_{max} \cdot \frac{t}{T_{warmup}} \quad (19)$$

Cosine Decay Phase ($t \geq T_{warmup}$): Let progress ratio $\tau = \frac{t - T_{warmup}}{T_{total} - T_{warmup}}$.

$$\eta_t = \eta_{min} + \frac{1}{2}(\eta_{max} - \eta_{min})(1 + \cos(\pi \cdot \tau)) \quad (20)$$

3.2.3 Code Implementation

```
1 def _create_scheduler(self):
2     num_training_steps = len(self.train_dataloader) * self.config.num_epochs //
3     self.config.gradient_accumulation_steps
4     num_warmup_steps = int(num_training_steps * self.config.warmup_ratio)
5
6     def lr_lambda(current_step):
7         if current_step < num_warmup_steps:
8             return float(current_step) / float(max(1, num_warmup_steps))
9
10        progress = float(current_step - num_warmup_steps) / float(max(1,
11        num_training_steps - num_warmup_steps))
12
13        return max(0.1, 0.5 * (1.0 + torch.cos(torch.tensor(3.14159) * progress)))
14    ))
```

```
12  
13     return LambdaLR(self.optimizer, lr_lambda)
```